Rapport : Projet de Programmation Numérique
Méthodes Ensemblistes pour la Modélisation

M1 CHPS

GUERROUDJ Djohra
LAKHANI Naïla
OUNNAS Masten
ZAMOUM Tassadit

Encadrant : JAM Mathys

Avril 2026

Table des matières

1 Introduction **2**
- 1.1 Rappel du sujet . . . 2
- 1.2 Rappel de la première partie . . . 2
- 1.3 Problématique . . . 2
- 1.4 Travail réalisé . . . 3

2 Profil Séquentiel **3**
- 2.1 Hardware . . . 3
- 2.2 Analyse . . . 3

3 Optimisation Algorithmique **5**
- 3.1 Choix des flags . . . 5
- 3.2 Méthode des Histogrammes . . . 5
- 3.3 Implémentation . . . 6
- 3.4 Passage de AoS à SoA [5] . . . 7
- 3.5 Validation . . . 7
  - 3.5.1 Implémentation naïve . . . 7
  - 3.5.2 Bagging . . . 8
  - 3.5.3 Boosting . . . 8

4 Optimisation Parallèle **9**
- 4.1 Environnement Technique et Topologie Matérielle . . . 9
- 4.2 Étude de Scalabilité (Strong Scaling) . . . 9
  - 4.2.1 Analyse du trade-off et de l'Overhead . . . 9
  - 4.2.2 Mesures sur le cluster Zen de Mesonet . . . 10
- 4.3 Comparaison avec l'État de l'Art . . . 11
  - 4.3.1 Benchmark du Bagging (Forêt Aléatoire) . . . 11
  - 4.3.2 Benchmark du Boosting (Gradient Boosting) . . . 11

5 Conclusion **12**

1 Introduction

1.1 Rappel du sujet

Ce projet est une étude des **méthodes ensemblistes pour la modélisation**, on s'intéresse plus particulièrement aux **arbres de décision** ainsi que leur optimisation grâce à des méthodes telles que le **Bagging** et le **Boosting**. Nous avons à notre disposition cinq jeu de données regroupant les temps d'exécution mesurés d'un programme de **factorisation LU** (DGETRF) :

- **15k_ga_adaptive.csv** qui contient 15 000 mesures générées par un algorithme génétique adaptatif. Les données suivent un pattern structuré, permettant aux modèles d'apprendre facilement et de produire de bonnes prédictions avec peu de bruit
- **15k_hvs.csv** qui contient 15 000 mesures également. Ce jeu de données possède beaucoup de bruit et de valeurs abérrantes (temps = 30s).
- **15k_random.csv** qui contient 15 000 mesures générées aléatoirement sans pattern prévisible. Ce jeu de donnée permet de mesurer la capacité des modèles à gérer des données imprévisibles et de comprer la performance sur des données non structurées
- **30k_ga_adaptive.csv** qui contient 30 000 mesures générées de la même manière que `15k_ga_adaptive.csv`.
- **random_1_million_filtered.csv** qui contient 1 million de mesures. Ce jeu de données nous sera utile pour pouvoir tester la scalabilité de notre implémentation lors de l'utilisation de clusters.

Ces temps d'exécutions dépendent de 8 différent leviers d'optimisation du noyau, dont les noms ont étés délibérémment obfusqués ainsi que des dimensions de la matrice factorisée.

L'objectif du premier semestre à été de mettre en oeuvre une **implémentation** de ces arbres et méthodes, il était donc nécessaire d'étudier l'impact des différentes générations de ces jeu de données sur nos performances. Cependant, l'objectif principal de ce deuxième semestre est **d'optimiser** cette implémentation, notamment grâce à la parallélisation. Pour cette raison, nous avons décidé de concaténer ces 4 premiers jeu de données en un fichier combined.csv pour le restant du projet.

1.2 Rappel de la première partie

La première phase du projet a été consacrée à la mise en œuvre d'une version fonctionnelle des algorithmes étudiés. Cette étape a conduit à **l'implémentation d'un arbre de décision** puis d'un **processus de filtrage** de données abérrantes, résultant à une diminution significative de l'erreur. Nous avons observé cela avec le jeu de donnée **15k_hvs.csv** qui contenait 255 valeurs abérrantes nous sommes passés de `RMSE = 3.29561` à `RMSE = 0.0971736` ce qui correspond à une réduction d'erreur de facteur 33,9.

Par ailleurs, deux approches ensemblistes ont été développées. La première, fondée sur le **bagging**, consiste à agréger plusieurs modèles indépendants entraînés sur des échantillons bootstrap des données, afin de réduire la variance du modèle global. La seconde, basée sur le **boosting**, repose sur une construction itérative de modèles faibles, chaque itération cherchant à corriger les erreurs des précédentes.

Bien que ces implémentations aient permis d'obtenir des résultats satisfaisants en termes de précision, elles ont été conçues sans considération particulière pour l'efficacité computationnelle.

1.3 Problématique

L'analyse de cette première version met en évidence plusieurs **limitations majeures**. D'une part, l'absence de **parallélisation** empêche toute exploitation des architectures multi-cœurs modernes, ce qui conduit à des temps d'exécution élevés. D'autre part, certaines parties de l'algorithme, notamment la recherche des meilleurs seuils de séparation, présentent une complexité algorithmique importante qui devient rapidement prohibitive lorsque la taille des données augmente.

Ces limitations se traduisent concrètement par une faible capacité de passage à l'échelle, tant en termes de volume de données que de ressources de calcul. **La problématique principale de cette seconde partie consiste donc à réduire ces coûts computationnels tout en préservant la qualité des modèles produits.**

1.4 Travail réalisé

Afin de répondre à ces enjeux, le travail mené s'est structuré en **plusieurs étapes** complémentaires. Une première phase a consisté en une **analyse** détaillée du comportement du programme en mode séquentiel, dans le but d'identifier précisément les zones critiques. Sur cette base, des **optimisations algorithmiques** ont été introduites, notamment à travers l'utilisation de structures de type histogramme permettant de réduire le coût de la recherche des splits. Enfin, une **stratégie de parallélisation** a été mise en place avec OpenMP.

L'ensemble de ces améliorations a permis d'obtenir des gains de performance significatifs, pouvant atteindre plusieurs facteurs multiplicatifs selon les configurations étudiées, tout en maintenant un niveau de précision comparable à celui de l'implémentation initiale.

2 Profil Séquentiel

2.1 Hardware

Voici la machine locale utilisée pour nos mesures de performance. Nous appelerons cette machine `[Laptop]` tout au long du rapport.

- **Modèle :** Lenovo IdeaPad Slim 3 (ordinateur portable)
- **Architecture :** x86_64
- **Processeur :** AMD Ryzen 5 7520U with Radeon Graphics
- **Fréquence du processeur :** 4,3 GHz
- **RAM :** 16 Go
- **Cœurs :** 4
- **Threads :** 8 (2 par cœur)
- **Cache :**
  - **L1d :** 128 Kb (par cœur)
  - **L1i :** 128 Kb (par cœur)
  - **L2 :** 2 Mb (par cœur)
  - **L3 :** 4 Mb

2.2 Analyse

Nous avons tout d'abord fait un rapport de perfomance de notre implémentation séquentielle d'arbre de régression naïf, voici les résultats obtenus :

Résultats de l'éxecution de l'implémentation séquentielle de construction d'arbre naïve sur la machine `[Laptop]` :

Modèle	RMSE	MAPE (%)	Temps (s)
Arbre Naïf (D=5)	0.063340	43.539806	0.695463
Arbre Naïf (D=8)	0.051227	26.368385	0.942628
Arbre Naïf (D=10)	0.048197	21.088756	0.757976
Arbre Naïf (D=15)	0.047214	16.821502	0.977817

TABLE 1 – `[Laptop]` Jeu de donnée combined.csv : 74719 lignes chargées (Filtrées=281,Seuil=26)

Commençons par une analyse visuelle du profil de l'implémentation avec un flamegraph modelisé avec le logiciel Hotspot :

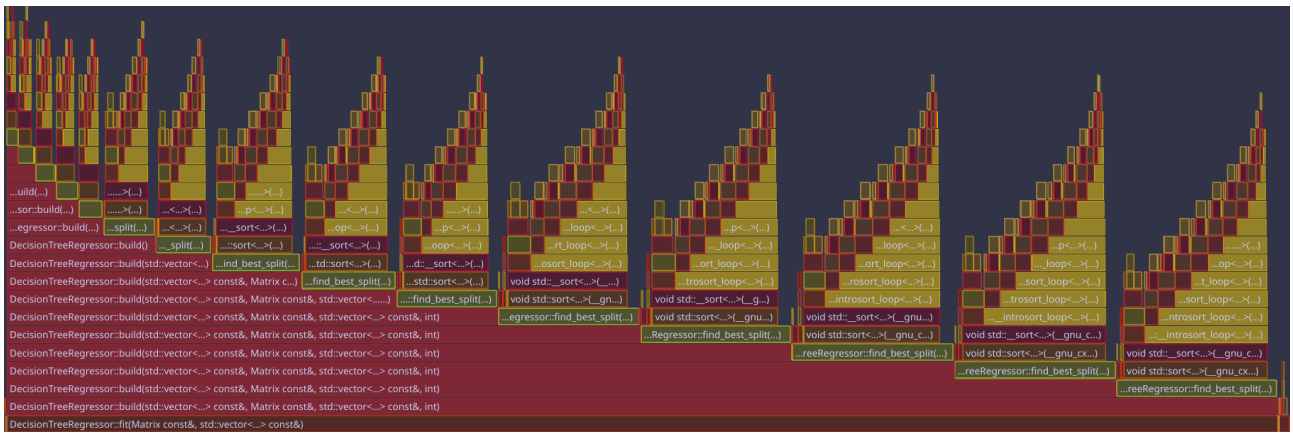


FIGURE 1 – [Laptop] Flamegraph de l'implémentation séquentielle de construction d'arbre naïve

Il met en évidence une structure d'appel récursive de fonctions. Pour chaque feature, chaque noeud, à chaque profondeur cette séquence est exécutée. Le tri est refait à chaque split. Le goulôt d'étranglement se trouve clairement lors de la recherche de split, plus spécifiquement la fonction sort. Le pourcentage CPU des fonctions les plus coûteuses nous permet de confirmer cela :

Fonction	**Pourcentage CPU**	**Rôle**
`operator`	$\sim 34.7\%$	Comparaison lors du tri
`__normal_iterator::operator++`	$\sim 9.9\%$	Itération sur les données
`__normal_iterator::operator-`	$\sim 9.4\%$	Calcul d'indices
`find_best_split`	$\sim 7.5\%$	Recherche du meilleur split
`operator`	$\sim 6.3\%$	Comparaisons supplémentaires

TABLE 2 – [Laptop] Répartition de l'utilisation CPU par fonction de l'implémentation d'arbre regréssif naïf

La fonction `operator` compare les valeurs lors du tri, ce n'est pas cette fonction en elle-même qui est coûteuse mais plutôt le fait qu'elle soit utilisé par `sort` aussi souvent.

Cette étape de recherche de split implique l'évaluation d'un grand nombre de seuils potentiels pour chaque variable, ce qui nécessite de multiples parcours des données.

D'un point de vue théorique, la complexité de cette opération peut être approximée par une expression de la forme

$$\mathcal{O}(n \cdot d \cdot k),$$

où n désigne le nombre d'exemples, d le nombre de variables et k le nombre de seuils testés. Cette complexité devient rapidement problématique lorsque les données sont volumineuses ou de grande dimension.

On retrouve des résultats similaires pour les implémentations séquentielles du **bagging** et du **boosting** puisqu'elles utilisent les mêmes fonctions de split :

On conclut de cette première analyse que les comparaisons coûteuses limite les performances de cette implémentation et que l'organisation des données en mémoire ne favorise pas la localité cache, c'est immédiatemment visible grâce à un perf stat qui nous indique 22.81% de cache-misses au total pour l'implémentation d'arbre, 23.96% pour le Bagging et 18.12% pour le Boosting.

Fonction	Pourcentage CPU	Rôle
`std::__introsort_loop<...>`	$\sim 61\%$	Tri des données
`build<...>`	$\sim 33.2\%$	Construction de l'arbre

TABLE 3 – `[Laptop]` Répartition de l'utilisation CPU par fonction de l'implémentation du bagging séquentiel

Fonction	Pourcentage CPU	Rôle
`std::__introsort_loop<...>`	$\sim 66.9\%$	Tri des données
`build<...>`	$\sim 28.5\%$	Construction de l'arbre

TABLE 4 – `[Laptop]` Répartition de l'utilisation CPU par fonction de l'implémentation du boosting séquentiel

3 Optimisation Algorithmique

3.1 Choix des flags

Lors du premier semestre, les mesures de performance ont toutes étés faites en utilisant les flags de compilation `-O3` et `-march=native`, cependant il n'y a pas eu d'analyse pour déterminer si ces flags sont réellement les plus adaptés à notre code. Nous profitons de ce deuxième semestre plus axé sur l'optimisation pour effectuer un benchmark complet des différents flags de compilation ainsi que de leur impact sur le temps d'éxecution de l'implémentation.

Flags	Time (s)	Speedup	Gain (%)
-O0	3.123242	1.000	0.00
-O2	1.091610	2.862	65.05
-O3	1.083882	2.883	65.30
-O3 -march=native	1.046276	2.986	66.50
-O3 -march=native -flto	1.049108	2.978	66.40

TABLE 5 – `[Laptop]` Temps d'exécution, speedup et gain en pourcentage par rapport à -O0

Les mesures sont obtenues en faisant la moyenne de 10 runs sur l'implémentation naïve d'arbre de régression de profondeur 15. Elles confirment notre choix du premier semestre. Par ailleurs, nous avons également decidé d'ajouter le flag `-flto` qui permet au compilateur d'optimiser le temps de liaison entre les fichiers [1]. Notre implémentation utilisant beaucoup de fonctions dans différents fichiers .cpp, ce flag pourra nous être utile. Cependant, cette optimisation n'est pas visible dans nos mesures puisque nous avons testé l'impact de ces flags sur l'implémentation de l'arbre de régression qui ne requiert pas tant de fichiers.

3.2 Méthode des Histogrammes

Afin de remédier aux limitations identifiées, une optimisation majeure a été introduite sous la forme de la **méthode des histogrammes** [2]. Cette approche, utilisée par des implémentations telles que `LightGBM` vise à réduire le nombre de seuils évalués lors de la recherche des meilleurs splits, en remplaçant une exploration exhaustive par une approximation contrôlée.



FIGURE 2 – Schéma illustrant la méthode des histogrammes [3]

Le principe repose sur une discrétisation des variables continues en un nombre fini d'intervalles, appelés **bins**. Chaque valeur est ainsi associée à un intervalle, ce qui permet de regrouper les données et de calculer des statistiques agrégées au niveau de ces bins. La recherche du meilleur split ne s'effectue plus sur l'ensemble des valeurs possibles, mais uniquement sur les frontières entre ces intervalles.

3.3 Implémentation

Cette transformation permet de **réduire significativement** le coût de calcul, en remplaçant une dépendance au nombre de données par une dépendance au nombre de bins, généralement beaucoup plus faible. L'implémentation repose sur une phase initiale de discrétisation, suivie de la construction d'histogrammes pour chaque variable. Les critères de séparation sont ensuite évalués à l'aide d'un balayage cumulatif, permettant de déterminer efficacement le point de coupure optimal.

Afin de déterminer quelle taille de bins est le plus optimal, nous effectuons un benchmark :

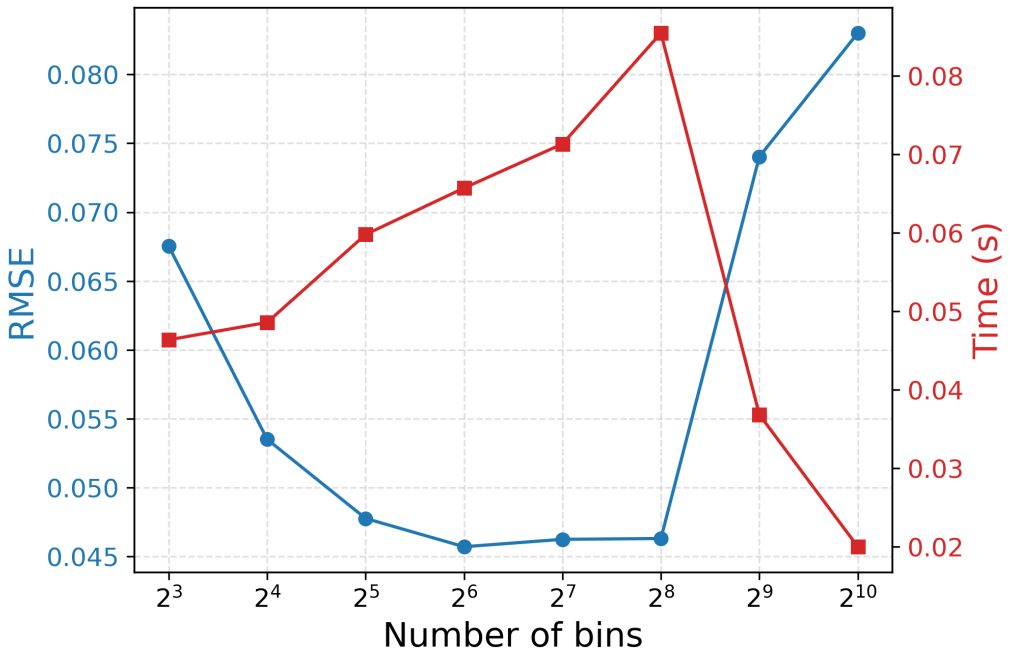


FIGURE 3 – [`Laptop`] Graphe comparant l'impact des tailles de bins sur le temps et l'erreur (arbre naïf de profondeur 15)

Par soucis de compromis, nous optons pour l'option `bins = 256`, même si le temps est plus grand

que pour `64` et `128`, c'est la valeur choisie par défaut par les implémentations existantes [4]. Pour faciliter la comparaison plus tard, on opte pour cette option avec l'ambition que ce temps s'améliorera grâce à la parallèlisation plus tard.

3.4 Passage de AoS à SoA [5]

En complément de l'optimisation algorithmique, une amélioration importante a été apportée à la structure des données en mémoire. L'implémentation initiale repose sur une organisation de type *Array of Structures* (AoS). Si cette approche est intuitive et simple à manipuler, elle présente des limites en termes de performance, notamment lors des accès séquentiels à une seule variable.

Afin d'améliorer la localité des accès mémoire et de réduire les cache misses, une transformation vers une organisation de type *Structure of Arrays* (SoA) a été effectuée. Dans ce modèle, chaque variable est stockée dans un tableau contigu indépendant, ce qui permet un accès plus efficace lors des parcours intensifs sur une seule feature.

Ce changement améliore significativement la performance des boucles critiques, en particulier lors du calcul des histogrammes et de la recherche des seuils de séparation. Il permet également une meilleure vectorisation par le compilateur et une réduction des accès mémoire non contigus, contribuant ainsi à une diminution globale du temps d'exécution.

3.5 Validation

Voici les résultats de cette première optimisation de performance :

3.5.1 Implémentation naïve

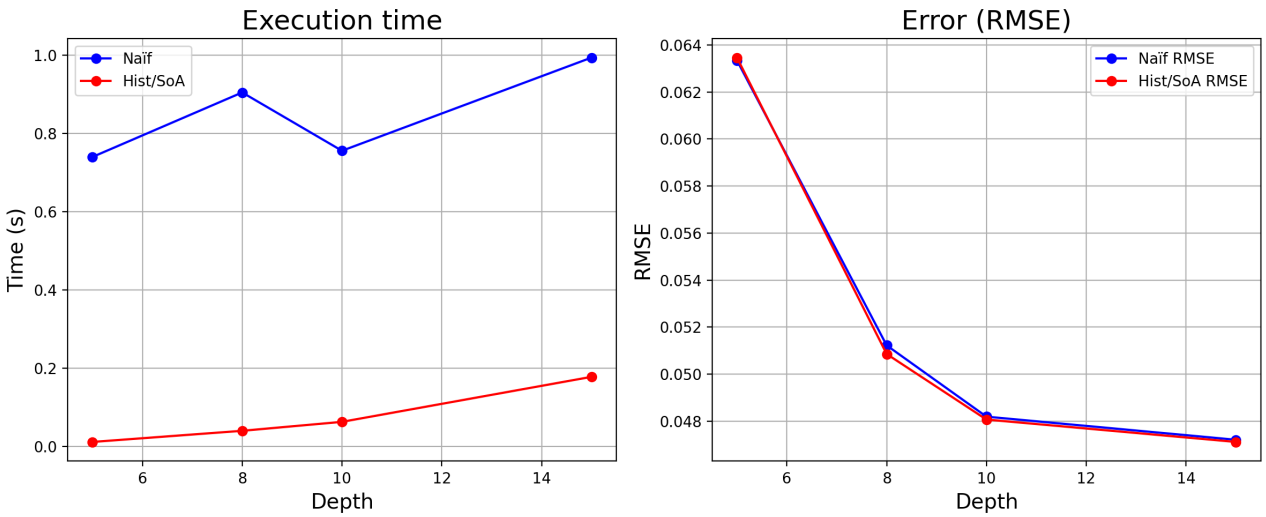


FIGURE 4 – [Laptop] Comparaison entre l'implémentation de contruction d'arbre naïve (S1) et l'implémentation histogramme/SoA

Le gain moyen est d'environ 18.8× (moyenne géométrique), avec un maximum à faible profondeur (65.5×) et une diminution progressive lorsque la profondeur augmente.

3.5.2 Bagging

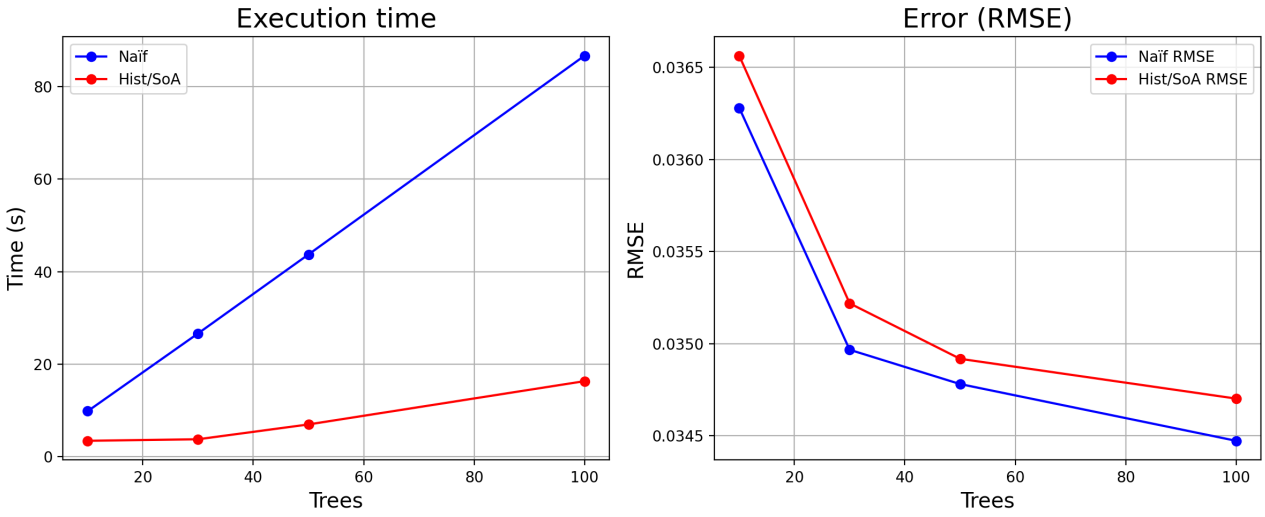


FIGURE 5 – [Laptop] Comparaison entre l'implémentation du Bagging naïf (S1) et l'implémentation histogramme/SoA

La méthode histogramme introduit une très légère dégradation de la précision (inférieure à 1% en RMSE), mais celle-ci est compensée par le gain en temps d'exécution : on a un speed-up moyen de 5.02x (moyenne géométrique).

3.5.3 Boosting

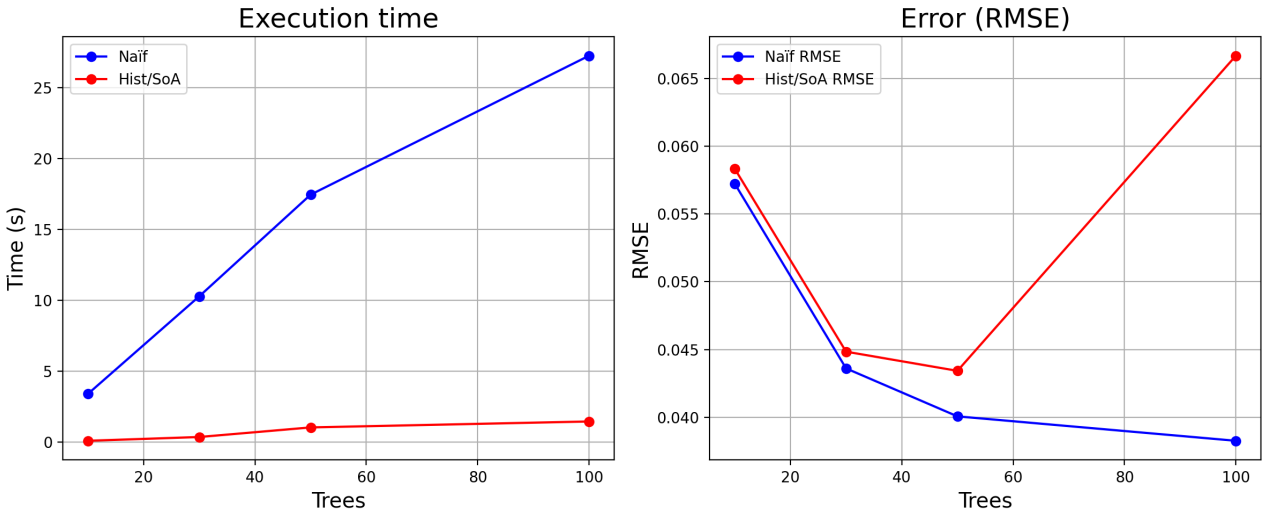


FIGURE 6 – [Laptop] Comparaison entre l'implémentation du Bagging naïf (S1) et l'implémentation histogramme/SoA

Le gain de performance est réel (24×), mais les résultats à 100 estimateurs suggèrent un problème dans l'implémentation optimisé du boosting, car la RMSE augmente anormalement alors que la tendance devrait être stable ou monotone.

4 Optimisation Parallèle

Au-delà des optimisations algorithmiques, l'exploitation du parallélisme constitue un levier essentiel pour améliorer les performances. Les architectures modernes offrent en effet des capacités importantes en termes de calcul parallèle, qu'il convient de mobiliser efficacement.

Dans le cas du bagging, la parallélisation est particulièrement naturelle, dans la mesure où les différents modèles sont indépendants les uns des autres. Il est ainsi possible de construire plusieurs arbres simultanément, sans nécessiter de synchronisation complexe. Cette propriété permet une distribution efficace de la charge de calcul, aussi bien sur des architectures multi-cœurs que sur des environnements distribués.

Le boosting, en revanche, présente des contraintes plus fortes en raison de sa nature intrinsèquement séquentielle. Chaque itération dépend des résultats de la précédente, ce qui limite les possibilités de parallélisation. Néanmoins, certaines parties du calcul, telles que la construction individuelle des arbres ou le traitement des données, peuvent être parallélisées localement.

Dans ce projet, on exploite l'interface OpenMP [6] afin d'optimiser les boucles de notre implémentation.

4.1 Environnement Technique et Topologie Matérielle

La compréhension de la hiérarchie mémoire est fondamentale pour interpréter les gains de performance obtenus. Les mesures ont été réalisées sur l'hôte `Suika-ASUS` (AMD Ryzen 7, 8 cœurs physiques / 16 threads logicaux).

— **Caches L1/L2 :** 32 KB / 512 KB dédiés par cœur.
— **Cache L3 :** 16 MB partagés pour l'ensemble du package.
— **Chaîne de compilation :** `g++` 13.x (`-O3 -march=native -flto -fopenmp`).

4.2 Étude de Scalabilité (Strong Scaling)

Nous avons mesuré l'impact de la parallélisation en faisant varier le nombre de threads OpenMP (`OMP_NUM_THREADS`). Cette étude permet d'identifier le seuil d'efficacité de l'implémentation S_2 sur le dataset `combined.csv` (74 719 lignes).

Modèle (N=100)	Naïf (S_1)	HPC 1 Th	HPC 2 Th	HPC 4 Th	HPC 8 Th
Bagging	38.69 s	2.33 s	**1.40 s**	2.27 s	5.25 s
Boosting	11.98 s	0.44 s	0.27 s	**0.24 s**	0.25 s

TABLE 6 – [`Suika-ASUS`] Temps d'exécution comparés selon le degré de parallélisme.

4.2.1 Analyse du trade-off et de l'Overhead

L'analyse des résultats met en évidence des comportements distincts selon l'algorithme :

— **Bagging (Saturation) :** Le temps d'exécution minimal est atteint avec **2 threads** (1.40 s). Au-delà, nous observons une dégradation brutale des performances (5.25 s à 8 threads). Ce phénomène s'explique par la méthode des histogrammes qui rend la construction de chaque arbre si rapide que l'overhead de gestion des threads OpenMP et la contention sur le cache L3 partagé deviennent prédominants par rapport au calcul réel.
— **Boosting (Stagnation) :** Le gain est visible jusqu'à **4 threads** (0.24 s) avant de stagner. La nature séquentielle du boosting (arbres dépendants) limite la parallélisation au calcul interne des histogrammes. Une fois ce calcul optimisé, le temps d'exécution atteint un plateau imposé par la partie séquentielle du code (Loi d'Amdahl).

4.2.2 Mesures sur le cluster Zen de Mesonet

Dans le cadre de ce projet, nous avons eu accès à un cluster afin d'effectuer des exécutions parallèles. Toutes les mesures ont été réalisées sur [Zen] :

— Un nœud du cluster Zen (MesoNET)
— En allocation exclusive via SLURM (–cpus-per-task=128 –exclusive)
— Chaque nœud est un système bi-socket AMD Zen totalisant 128 cœurs (2 sockets × 64 cœurs)
— Le programme a été compilé avec GCC 13.2.0 avec les options -O3 -flto -fopenmp
— L'affinité des threads a été fixée par OMP_PROC_BIND=close et OMP_PLACES=cores
— Le dataset utilisé est random_1M.csv (991 723 points, 10 features), filtré sur le seuil de performance à 26.0

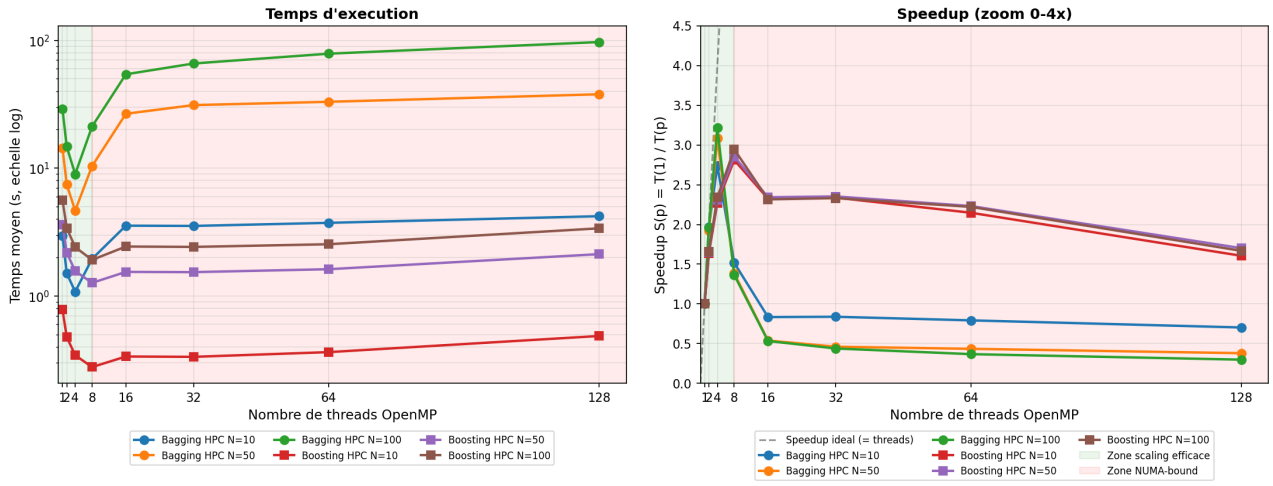


FIGURE 7 – [Zen] Scalabilité forte OpenMP sur 128 threads

On se rend compte que l'exécution de l'implémentation sur un plus grand nombre de threads n'améliore pas pour autant la parallèlisation.

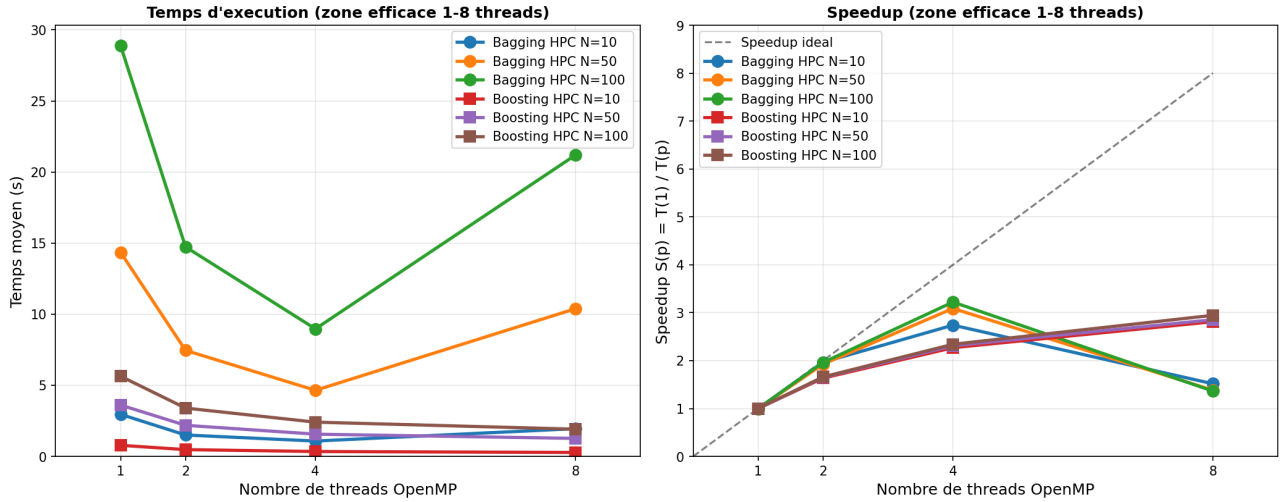


FIGURE 8 – [Zen] Scalabilité forte OpenMP sur 8 threads

Lorsque l'on observe les résultats sur 8 threads, on observe un speed-up qui se rapproche du speed-up idéal : on peut dire que l'implémentation est "scalable" jusqu'à 4 threads et que la parallèlisation

est efficace mais après cela, le speed-up baisse drastiquement :on observe un "knee point" à partir de 4 threads, le speed-up baisse même en-dessous de 1 pour le Bagging, le rendant plus lent que la version séquentielle, ce qui est anormal. Hypothèses :

— Saturation de la mémoire bande passante, puisque les threads OMP partagent la même mémoire, ils se battent pour la bande passante
— Pour le Bagging : il crée trop de modèles indépendants, surtout sur un grand jeu de donnée, ce qui demande un accès mémoire coûteux
— L'idéal n'est jamais atteint, ce qui est expliqué par la Loi d'Amdahl : le speed-up va éventuellement stagner selon le pourcentage de parallèlisation du code même si on augmente le nombre de threads
— Les overhead d'OpenMP ralentissent la performance

4.3 Comparaison avec l'État de l'Art

Pour évaluer la pertinence de nos optimisations, nous comparons les performances "crêtes" de notre implémentation S_2 aux bibliothèques de référence. Chaque modèle est configuré avec des hyperparamètres strictement identiques pour garantir l'équité du benchmark.

4.3.1 Benchmark du Bagging (Forêt Aléatoire)

Nous comparons ici notre meilleure configuration de Bagging (obtenue avec 2 threads) à l'implémentation `RandomForestRegressor` de Scikit-Learn.

Implémentation	**Paramètres de test**	**RMSE**	**Temps (s)**
Projet HPC (S_2)	$N = 100, D = 12, \text{bins} = 256, \text{threads} = 2$	0.0347	**1.40 s**
Scikit-Learn (RF)	$N = 100, D = 12, \text{n_jobs} = -1$	0.0163	1.88 s

TABLE 7 – [`Suika-ASUS`] Comparaison des performances optimales du Bagging.

Analyse : Notre implémentation spécialisée parvient à surpasser Scikit-Learn en temps d'exécution (gain de $\approx 25\%$). Ce résultat souligne l'efficacité de la structure SoA et du calcul par histogrammes qui réduit drastiquement le coût de recherche des splits par rapport au tri exact utilisé par défaut dans certaines bibliothèques généralistes.

4.3.2 Benchmark du Boosting (Gradient Boosting)

Le tableau suivant compare notre version de Boosting la plus performante (4 threads) aux moteurs industriels basés sur les histogrammes.

Moteur	**Configuration ($N = 100, D = 4, \textbf{LR} = 0.2$)**	**RMSE**	**Temps (s)**
Projet HPC (S_2)	**bins=256, threads=4**	0.0666	**0.247 s**
XGBoost (Hist)	`tree_method='hist'`, `max_bin=256`	0.0309	0.096 s
Sklearn (Hist)	`max_bins=255`	0.0318	0.126 s
LightGBM	`max_bin=255`, `n_jobs=-1`	0.0312	0.362 s

TABLE 8 – [`Suika-ASUS`] Benchmark industriel du Boosting sur 74 719 lignes.

Analyse : Sur ce volume de données, notre code est environ 1.4× plus rapide que **LightGBM**. Bien que **XGBoost** conserve l'avantage en vitesse pure et en précision, l'écart de RMSE (≈ 0.03) est le compromis accepté pour notre méthode de *binning* uniforme simplifiée, optimisée pour la bande passante mémoire.

5 Conclusion

Ce travail a permis de démontrer l'importance des optimisations algorithmiques et de la parallélisation dans le contexte des algorithmes d'apprentissage automatique. L'utilisation de la méthode des histogrammes a permis de réduire significativement la complexité des calculs, tandis que la parallélisation a offert des gains substantiels en temps d'exécution mais seulement pour un nombre de threads limité.

Malgré ces limitations, des améliorations significatives ont pu être apportées. Des perspectives d'amélioration subsistent, notamment en ce qui concerne l'exploitation des architectures GPU, l'optimisation fine de la gestion mémoire ou encore l'exploration de stratégies hybrides de parallélisation. Ces axes constituent des pistes prometteuses pour des travaux futurs.

Références

[1] GNU Project. *Optimize Options (Using the GNU Compiler Collection (GCC))*. Accessed : 2026-05-06. 2024. URL : https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html.

[2] In : *ScienceDirect* (2021). Accessed : 2026-05-06. DOI : 10.1016/S1047-3203(21)00050-X. URL : https://www.sciencedirect.com/science/article/abs/pii/S104732032100050X.

[3] "Article on histogram-based gradient boosting". In : *ScienceDirect* (2023). Accessed : 2026-05-06. URL : https://www.sciencedirect.com/science/article/abs/pii/S0926580523000274.

[4] XGBoost Developers. *XGBoost Parameters Documentation*. Accessed : 2026-05-06. 2024. URL : https://xgboost.readthedocs.io/en/stable/parameter.html.

[5] Hugo Taboada. *CHP203 TOP Course 2025*. Course material (PDF), Accessed : 2026-05-06. 2025. URL : https://dssgabriel.github.io/CHP203-TOP/artifacts/TOP_course_2025.pdf.

[6] OpenMP Architecture Review Board. *OpenMP Reference Guides*. Includes OpenMP 6.0 (Nov 2024) and earlier reference guides ; Accessed : 2026-05-06. 2024. URL : https://www.openmp.org/resources/refguides/.

[7] F. Pedregosa et al. "Scikit-learn : Machine Learning in Python". In : *Journal of Machine Learning Research* 12 (2011), p. 2825-2830.

[8] Dimitris Bertsimas et Berk Öztürk. "Global Optimization via Optimal Decision Trees". In : *Journal of Global Optimization* (2023). DOI : 10.1007/s10898-023-01311-x.

[9] Stéphane Caron. *Une introduction aux arbres de décision*. n.d.

[10] A. C. Davison et S. Sardy. *Méthodes de rééchantillonnage pour l'estimation de variance en sondage*. n.d.

[11] Yannig Goude. *Arbres de Régression, CART*. n.d.

[12] Gavin Hackeling. *Mastering Machine Learning with Scikit-Learn*. Packt Publishing, 2014.

[13] Guolin Ke et al. *LightGBM : A Highly Efficient Gradient Boosting Decision Tree*. n.d.

[14] Han Liu et Mihaela Cocea. "Semi-Random Partitioning of Data into Training and Test Sets in Granular Computing Context". In : *Granular Computing* 2.4 (2017), p. 357-386. DOI : 10.1007/s41066-017-0049-2.

[15] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. n.d.

[16] Bernard M. E. Moret. "Decision Trees and Diagrams". In : *ACM Computing Surveys* 14.4 (1982), p. 593-623. DOI : 10.1145/356893.356898.

[17] S. K. Murthy, S. Kasif et S. Salzberg. "A System for Induction of Oblique Decision Trees". In : *Journal of Artificial Intelligence Research* 2 (1994), p. 1-32. DOI : 10.1613/jair.63.

[18] J. P. Nakache. *Méthode de discrimination basée sur la construction d'un arbre de décision binaire*. n.d.

[19] Mehrbakhsh Nilashi, Rabab Ali Abumalloh, Behrouz Minaei-Bidgoli et al. "Predicting Parkinson's Disease Progression : Evaluation of Ensemble Methods in Machine Learning". In : *Journal of Healthcare Engineering* (2022), p. 1-17. DOI : 10.1155/2022/2793361.

[20] Paweł Piotrowski et al. "Advanced Ensemble Methods Using Machine Learning and Deep Learning for One-Day-Ahead Forecasts of Electric Energy Production in Wind Farms". In : *Energies* 15.4 (2022), p. 1252. DOI : 10.3390/en15041252.

[21] J. R. Quinlan. "Induction of Decision Trees". In : *Machine Learning* 1.1 (1986), p. 81-106. DOI : 10.1007/BF00116251.

[22] J. Ross Quinlan. *C4.5 : Programs for Machine Learning*. Morgan Kaufmann, 2014.

[23] J. Ross Quinlan. *Inferring Decision Trees Using the Minimum Description Length Principle*. n.d.

[24] Sebastian Raschka. *STAT 451 : Machine Learning Lecture Notes*. 2020.

[25] Yan-yan Song et Ying Lu. "Decision Tree Methods : Applications for Classification and Prediction". In : *Statistics* 27.2 (2015).

[26] AboutTrading. *Histogram-Based Gradient Boosting*. Accessed : 2026-05-06. 2023. URL : https://abouttrading.substack.com/p/histogram-based-gradient-boosting.

[27] "Histogram-based Gradient Boosting (Book Chapter)". In : *Lecture Notes in Computer Science*. Accessed : 2026-05-06. Springer, 2020. DOI : 10.1007/978-3-030-37334-4_4. URL : https://dl.acm.org/doi/10.1007/978-3-030-37334-4_4.

[28] GeeksforGeeks. *HistGradientBoostingClassifier in Scikit-learn*. Accessed : 2026-05-06. 2024. URL : https://www.geeksforgeeks.org/machine-learning/histgradientboostingclassifier-in-sklearn/.

[29] MesoNET. *Accès SSH - Documentation utilisateur*. Accessed : 2026-05-06. 2024. URL : https://www.mesonet.fr/documentation/user-documentation/acces/ssh/.

[30] MesoNET. *Portail - Demander un compte*. Accessed : 2026-05-06. 2024. URL : https://www.mesonet.fr/documentation/user-documentation/acces/portail/#demander-compte.

[31] Shan Suthaharan. *Machine Learning Models and Algorithms for Big Data Classification*. T. 36. Integrated Series in Information Systems. Springer US, 2016. DOI : 10.1007/978-1-4899-7641-3.

[32] Thomas G. Dietterich. "Ensemble Methods in Machine Learning". In : *Multiple Classifier Systems*. Springer, 2000, p. 1-15. DOI : 10.1007/3-540-45014-9_1.

[33] Leo Breiman. "Bagging Predictors". In : *Machine Learning* 24.2 (1996), p. 123-140. DOI : 10.1007/BF00058655.

[34] Leo Breiman. "Random Forests". In : *Machine Learning* 45.1 (2001), p. 5-32. DOI : 10.1023/A:1010933404324.

[35] Jerome H. Friedman. "Greedy Function Approximation : A Gradient Boosting Machine". In : *Annals of Statistics* 29.5 (2001), p. 1189-1232. DOI : 10.1214/aos/1013203451.

[36] Tianqi Chen et Carlos Guestrin. "XGBoost : A Scalable Tree Boosting System". In : *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. 2016, p. 785-794. DOI : 10.1145/2939672.2939785.

[37] Guolin Ke et al. "LightGBM : A Highly Efficient Gradient Boosting Decision Tree". In : *Advances in Neural Information Processing Systems (NeurIPS)*. NeurIPS. 2017. URL : https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bb9d9e6b76fa-Abstract.html.

[38] Michael J. A. Berry et Gordon S. Linoff. *Mastering Data Mining : The Art and Science of Customer Relationship Management*. 1st. Wiley, 1999. ISBN : 978-0471246303.

[39] Pierre Geurts, Damien Ernst et Louis Wehenkel. "Extremely Randomized Trees". In : *Machine Learning* 63.1 (2006), p. 3-42. DOI : 10.1007/s10994-006-6226-1.

[40] Yoav Freund et Robert E. Schapire. "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting". In : *Journal of Computer and System Sciences* 55.1 (1997), p. 119-139. DOI : 10.1006/jcss.1997.1504.

[41] Analytics Vidhya. *How to Implement the AdaBoost Algorithm ?* Accessed : 6 mai 2026. 2021. URL : https://www.analyticsvidhya.com/blog/2021/09/adaboost-algorithm-a-complete-guide-for-beginners/.

[42] Data Science Wizards. *Understanding the AdaBoost Algorithm*. Accessed : 6 mai 2026. 2023. URL : https://medium.com/@datasciencewizards/understanding-the-adaboost-algorithm-2e9344a833d9.